Acemark Consulting

ARCHITECTURE DECISION RECORD

Vitalia Seguros.

PREPARADO POR
Acemark Consulting

FECHA
Lunes, 26 de octubre de 2026

VERSIÓN
Final

Architectural Decision Records (ADR)

Anexo al documento de Fase 2. Las ~70 decisiones del proyecto viven en los **registros por subfase** (AUT · SLA · SEC · IDE · INT · UX); aquí se desarrollan sólo las cinco que cumplen al menos uno de estos tres criterios:

Criterio	Por qué merece ADR
Irreversible en la práctica	Deshacerla implica migración de datos o reconfiguración estructural
Contestable en revisión	Un arquitecto razonable podría haber decidido lo contrario
Cambió de dirección	Se decidió una cosa, se revirtió, y la razón de la reversión importa

> **Un criterio que admite todo no es un criterio.** De siete candidatos, dos quedaron fuera — están al final, con el motivo.

Numeración por orden de creación, sin renumerar. El orden de lectura lógico no es el mismo que el de creación: ADR-01 se apoya en ADR-04, que se escribió después. Eso se resuelve con referencias cruzadas, no reordenando. Es la convención, y romperla rompe cualquier enlace externo al registro.

Índice

ADR	Decisión	Fase	Criterio
ADR-01	El rollup de HU-13 se implementa en Apex, no con Roll-Up Summary	2.4	Irreversible · Contestable
ADR-02	El endoso se registra en el core; Salesforce lo muestra	2.6	**Cambió de dirección**
ADR-03	Person Accounts activado, conviviendo con Business Accounts	2.1	**Irreversible a nivel de org**
ADR-04	Account en Public Read Only pese al aislamiento por unidad	2.2	Contestable
ADR-05	El re-login no actualiza los accesos por unidad	2.5	Contestable

Mapa de dependencias

ADR-03 Person Accounts
└→ ADR-04 Account en Public Read Only
└→ ADR-01 Rollup en Apex
(el OWD de Account prohíbe el master-detail,
el master-detail es la única vía al Roll-Up Summary)
ADR-02 Endoso en el core (independiente)
ADR-05 Accesos por unidad manuales (independiente)

ADR-04 es la raíz. Si cambia, ADR-01 deja de sostenerse.

ADR-01 · El rollup de HU-13 se implementa en Apex, no con Roll-Up Summary

Estado	Aceptada
Fase	2.4 — Diseño de automatizaciones
Decisión en el log	AUT-13, A1
Criterios que cumple	Irreversible en la práctica · Contestable en revisión
Decisiones relacionadas	**ADR-04** (OWD de Account), AUT-02 (patrón de handler), AUT-11 (eje de mantenibilidad)

Contexto

HU-13 pide que la cuenta muestre el **número de pólizas activas** y la **suma de primas** de su titular, con el criterio de aceptación «*el agente puede confiar en el dato sin verificarlo manualmente*».

El enunciado del caso señala explícitamente este requisito: «*Este es un buen candidato para resolverse con un Apex Trigger — evalúalo frente a la alternativa de Flow y documenta tu decisión*».

La formulación del enunciado induce un error: presenta la decisión como binaria (Flow o Apex). No lo es.

Alternativas evaluadas

1 · Roll-Up Summary — descartada

Es la primera que hay que descartar, no la última. Un Roll-Up Summary resuelve el requisito **sin una línea de código y sin coste de mantenimiento**, y sería la respuesta correcta si la relación lo permitiera.

Requiere master-detail. `Policy__c → Account` está modelada como lookup.

La pregunta real no es «¿por qué no un RUS?», sino **«¿por qué no convertir la relación en master-detail para poder usarlo?»**. Dos razones, y la primera es de seguridad:

> 🔴 **Master-detail fuerza al hijo a heredar el OWD del padre.**
> Account está en **Public Read Only** (**ADR-04**), por el requisito operativo de que cualquier agente pueda identificar a quien llama antes de saber de qué unidad es. Con master-detail, `Policy__c` pasaría a Public Read Only. Las pólizas serían **legibles por los 65 usuarios de la org**, y el aislamiento por unidad de negocio de **HU-09 quedaría destruido** — no debilitado: eliminado.

Segunda razón — borrado en cascada. Master-detail borra los hijos al borrar el padre. Con 10M pólizas replicadas del core, una eliminación de cuenta accidental o un proceso de depuración mal parametrizado destruiría datos que Salesforce no es fuente de verdad para restaurar (**ADR-02**).

2 · Record-Triggered Flow — viable, pero peor

Técnicamente posible. Falla en tres puntos:

Agregación	200 pólizas pueden pertenecer a 50 cuentas distintas. Hay que agrupar por cuenta, sumar, y actualizar cada cuenta **exactamente una vez**. En Flow eso son variables de colección y bucles anidados
El fallo de bulkificación	El error clásico es un *Get Records* dentro de un bucle. Aquí la tentación es directa, y el fallo no aparece en pruebas de un registro
Mantenibilidad invertida	El argumento habitual a favor de Flow — *«lo mantiene un administrador sin programar»* — **deja de aplicar**. Un Flow con tres bucles y dos colecciones no es más legible que una clase handler: es menos legible, y además esconde sus propios fallos de escala (AUT-11)

Y el **caso negativo de HU-13** — *«si otra automatización toca el mismo objeto, el resultado sigue siendo correcto y no se duplica la lógica»* — pide un punto de entrada único por objeto. Eso es el patrón trigger + handler (AUT-02), no un Flow adicional compitiendo por el mismo evento sin orden garantizado.

3 · Paquete declarativo de rollups — descartada

Resuelve rollups sobre lookup sin código. Descartada por **dependencia de un managed package para un requisito central** de la vista 360: una actualización del paquete, un cambio de licenciamiento o un fin de soporte comprometerían un criterio de aceptación.

4 · Apex Trigger + Handler — aceptada

`PolicyTrigger → PolicyTriggerHandler`, bulkificado, con **recálculo desde el estado actual** (AUT-08) en lugar de incremento. Un contador que suma y resta se desincroniza ante cualquier fallo parcial; uno que recalcula converge siempre — y hace que el criterio de rehabilitación de HU-19 salga gratis.

Decisión

Apex Trigger con handler separado. No por volumen — 200 registros por transacción los maneja cualquier herramienta — sino porque **el modelo de seguridad prohíbe la alternativa declarativa**.

~~«El enunciado lo exige»~~	Deferencia, no arquitectura
«Son 200 registros»	Cierto, pero secundario
«El modelo de seguridad prohíbe la alternativa declarativa»	✅

Consecuencias

Positivas

- HU-09 queda intacta: `Policy__c` conserva OWD Private y sus propias sharing rules
- Sin riesgo de borrado en cascada sobre 10M registros
- Punto de entrada único por objeto; el caso negativo de HU-13 se cumple por diseño

Negativas y aceptadas

- Requiere mantenimiento por un desarrollador, no por un administrador
- Obliga a prueba unitaria de **200 registros** (criterio obligatorio de HU-13) y a prueba del caso negativo

- Introduce el problema de la cascada en carga masiva, resuelto con bypass por Custom Permission y recálculo asíncrono (AUT-03), al precio de **consistencia eventual** tras la sincronización nocturna (AUT-04)

Por qué es irreversible en la práctica

Cambiar de opinión más adelante significa convertir un lookup con 10M registros en master-detail — migración de datos, reconfiguración completa del modelo de compartición de `Policy__c`, y revalidación de HU-09, HU-10 y HU-29. **No es una decisión que se revise en el sprint siguiente.**

El principio generalizable

> **Cuando una alternativa declarativa exige cambiar el modelo de relaciones, se evalúa primero contra el modelo de seguridad, no contra el volumen.**

Registrado como **AUT-13**. La relación entre objetos no es sólo una decisión de modelo de datos: en Salesforce es también una decisión de seguridad, porque master-detail transfiere el OWD.

Preguntas que un revisor hará

1. ¿Por qué `Policy__c → Account` es lookup y no master-detail? → Por ADR-04 y HU-09
2. ¿El rollup es exacto en todo momento? → No. Exacto en operación de usuario, eventualmente consistente tras el batch (AUT-04)
3. ¿Qué pasa si otra automatización modifica `Policy__c` en la misma transacción? → Punto de entrada único por AUT-02; cubierto por prueba del caso negativo
4. ¿Se evaluó algún paquete de rollups declarativos? → Sí, descartado por dependencia externa en requisito central

---

ADR-02 · El endoso se registra en el core; Salesforce lo muestra

Estado	Aceptada — **revierte una decisión anterior**
Fase	2.6 — Integración con el sistema legacy
Decisión en el log	INT-02, AUT-10
Criterios que cumple	**Cambió de dirección** · Contestable en revisión
Decisiones relacionadas	UX-06 (botón al core), A9 · I3 (refresco puntual), HU-19 (reescrita), INT-01 (fuente de verdad por entidad)

Contexto

HU-19 pedía originalmente que el agente **registrara un endoso** desde Salesforce: un cambio de cobertura, de prima o de datos sobre una póliza vigente.

El problema es dónde vive el motor de tarificación. **Está en el core.** Un endoso con efecto económico recalcula la prima, y ese cálculo no es replicable en Salesforce sin reconstruir reglas actuariales que nadie ha documentado.

Alternativas evaluadas

1 · Registrar el endoso en Salesforce — descartada tras haberse aceptado

Fue la **primera decisión**, y se aprobó con condiciones. El diseño que requería:

Pieza	Para qué
Flujo de confirmación en dos fases	Escribir en Salesforce, pedir al core, confirmar o revertir
Clase Apex de orquestación	Gestionar la llamada y el retorno
Cuatro estados de sincronización	Pendiente · confirmado · rechazado · divergente
Dos reglas de automatización	Reintento y marcado de divergencia

Dos problemas, y el segundo es el que la tumbó:

> **El agente no escribe la prima.** Si Salesforce no recibe la prima recalculada, alguien la teclea. Vitalia tendría entonces **dos precios para la misma póliza**, y el que ve la DGSFP es el del core. Eso no es calidad de dato: es **exposición regulatoria**.
> 🔴 **Y el diseño entero dependía de un supuesto no verificable:** que el core devolviera los valores recalculados en la misma llamada. **Nadie lo había confirmado con el proveedor.**

El segundo es el decisivo. No era un riesgo técnico acotable — era una dependencia sobre una capacidad que no se podía comprobar sin hablar con un tercero que no estaba en la sala.

2 · Registrar en el core; Salesforce de sólo lectura — aceptada

`Policy__c` y `Policy_Movement__c` pasan a **sólo lectura en Salesforce** (AUT-10). El agente sale al core para endosar y vuelve.

Decisión

El endoso se registra en el core. Salesforce lo muestra.

Dos piezas hacen que sea usable y no una degradación:

UX-06	Botón en `Policy__c` que abre esa misma póliza en el core — enlace profundo, no «entra y búscala»
A9 · I3	Botón «actualizar desde el core»: refresca la póliza **y sus movimientos** sin esperar a la sincronización nocturna

Sin I3, el agente endosaría y no vería su propio trabajo hasta el día siguiente. Esa es la diferencia entre una decisión y una excusa.

Lo que se compra y lo que se vende

	Registrar en Salesforce	Registrar en el core ✅
Riesgo que elimina	—	Dependencia de un supuesto no verificable
Riesgo que acepta	Divergencia del dato económico	**El agente sale de Salesforce para endosar**
Complejidad añadida	Flujo de confirmación + estados de sincronización	**Ninguna**

> **Es un cambio de riesgo técnico por riesgo de adopción.** El técnico no se podía acotar; el de adopción sí — con un botón.

Consecuencias

La reversión eliminó un flujo, una clase Apex, cuatro estados de sincronización y dos reglas de automatización.

> **Un diseño que se simplifica al quitar un supuesto suele ser el diseño correcto.**

HU-19 cambia de naturaleza: deja de ser una historia de escritura y pasa a ser *«consultar el histórico de endosos y su efecto en la póliza»*. Sus criterios de aceptación se reescriben, la estimación baja, y libera holgura en los sprints 1-2 — estaba en el grupo fundacional con HU-13, HU-16 y HU-27.

Efecto en 2.2: ningún perfil de negocio crea ni edita `Policy__c` ni `Policy_Movement__c`. Es una fila de la matriz CRUD, y necesita prueba explícita en Fase 3.

Efecto en el modelo de datos: da sentido al snapshot tipado de 2.1. `Prima_Resultante__c` lo calcula quien tiene el motor, no una fórmula de Salesforce con el tipo de cambio de hoy.

Riesgo vivo — RSK-INT-02: el agente endosa fuera de Salesforce y el dato puede registrarse tarde, o no registrarse. Mitigado por UX-06 e I3, no eliminado.

Por qué está escrito el historial

Un revisor preguntará **por qué el endoso no se registra donde trabaja el agente**. Es la pregunta correcta, y merece respuesta escrita en lugar de improvisada.

El valor de este ADR no es la decisión final: es que **hubo una anterior y se revirtió con motivo**. Un registro que sólo guarda el resultado no distingue entre una decisión pensada y una que salió bien por suerte.

Preguntas que un revisor hará

1. ¿Por qué no registrar el endoso donde trabaja el agente? → Prima recalculada por el core; dos precios para la misma póliza es exposición regulatoria ante la DGSFP
2. ¿Qué se intentó primero y por qué no funcionó? → Confirmación en dos fases; dependía de que el core devolviera los valores recalculados, supuesto no verificado
3. ¿El agente tiene que trabajar en dos sistemas? → Sí. Mitigado con enlace profundo (UX-06) y refresco bajo demanda (I3)
4. ¿Qué pasa si el agente endosa y no vuelve a Salesforce? → RSK-INT-02; el dato llega por la sincronización nocturna, con hasta 24 h de retraso
5. ¿Se verificó el supuesto con el proveedor? → **No.** Esa es precisamente la razón de la reversión

---

ADR-03 · Person Accounts activado, conviviendo con Business Accounts

Estado	Aceptada
Fase	2.1 — Modelo de datos
Decisión en el log	Modelo de datos · Account
Criterios que cumple	**Irreversible a nivel de organización**
Decisiones relacionadas	**ADR-04** (OWD de Account), SEC-05 (propiedad del registro), HU-01 · HU-03 (propiedad del lead y de la cartera)

Contexto

La cartera de Vitalia es **mayoritariamente particular** — en torno al 80%. El resto son empresas: pólizas de flota, multirriesgo de comercio, colectivos de vida.

Salesforce modela por defecto `Account` = organización y `Contact` = persona dentro de ella. Una aseguradora de particulares no encaja en ese molde: el titular de la póliza **es una persona**, y `Policy__c.Account__c` es un lookup obligatorio.

Alternativas evaluadas

1 · Sólo Business Accounts, con los particulares como Contact

El modelo estándar. Obliga a colgar `Policy__c` de `Contact` en lugar de `Account` — y entonces las pólizas de empresa no tienen dónde ir, porque una flota no pertenece a un contacto.

La variante habitual es peor: **una cuenta «Particulares» compartida** con millones de contactos colgando.

> 🔴 **Eso destruye el modelo de compartición entero.** Una cuenta tiene **un** propietario. HU-01 asigna el lead a un agente, HU-03 razona sobre *«sus»* registros, y toda la Capa 5 de 2.2 se apoya en la propiedad del registro. Con una cuenta compartida, los 65 usuarios tienen el mismo propietario para todos los particulares y la propiedad deja de significar nada.

2 · Objeto personalizado para particulares

Duplicaría el modelo: dos objetos cliente, dos modelos de compartición, dos lookups desde cada objeto hijo, y `Policy__c`con dos padres posibles. Se descarta sin más análisis.

3 · Person Accounts — aceptada

Un `Account` que representa a una persona. `Policy__c.Account__c` apunta a un único objeto sea el titular persona o empresa, y el modelo de propiedad funciona igual para los dos.

Decisión

Person Accounts activado, conviviendo con Business Accounts. Account lleva Record Types **Person · Business**.

🔴 Por qué este ADR existe: es irreversible

> **Activar Person Accounts no se puede deshacer.** Salesforce no ofrece una vía para desactivarlo una vez habilitado en la organización. No es «caro de revertir»: es **imposible**.

Es la única decisión del proyecto con esa propiedad. ADR-01 es caro de cambiar; éste no se cambia.

Por eso merece ADR aunque la alternativa sea claramente peor: **una decisión que no admite marcha atrás se documenta aunque parezca obvia**, porque dentro de dos años nadie recordará qué se evaluó.

Lo que cuesta, y que hay que aceptar por escrito

Consecuencia	Qué significa
Un Person Account consume dos registros	Account + el Contact oculto. Con millones de clientes, afecta al almacenamiento y al coste
Aparece en reportes de Account y de Contact	M1 y cualquier reporte de cartera necesitan criterio explícito para no contar dos veces
Record Types en Account	Habilitarlo **crea** el Record Type de Person Account. La columna de Record Types de Account no puede decir «—»
Carga de datos más compleja	I1 debe cargar con el Record Type correcto; el comportamiento de Bulk API difiere del de una cuenta normal
Compatibilidad de paquetes	Algunos paquetes de AppExchange no soportan Person Accounts. Es un hecho de plataforma a tener presente si alguna vez se evalúa uno — **no hay ninguno previsto en este diseño**

El punto abierto: dimensionar el almacenamiento

De las cinco consecuencias, **sólo una exige acción antes de Fase 3**: los dos registros por persona.

⚠ **Ø Sin dato.** El proyecto documenta **10M pólizas vigentes**; no documenta cuántos clientes. Con la cartera mayoritariamente particular, el número de Person Accounts es del mismo orden de magnitud, y cada uno cuenta doble.

Acción, no riesgo: cuantificar el volumen de clientes y dimensionar el almacenamiento **antes de I1**. No es un riesgo porque no hay incertidumbre sobre si ocurrirá — ocurre siempre; sólo falta el número.

> Las otras cuatro consecuencias son costes aceptados, no pendientes. Registrarlas como riesgos sería inflar el registro:* **un riesgo necesita un escenario que trace a un requisito**, y éstas trazan a una decisión ya tomada.

Qué habilita

- `Policy__c.Account__c` tiene un solo tipo de padre, sea el titular persona o empresa
- La propiedad del registro funciona igual para los dos — y la propiedad es el cimiento de toda la Capa 5 de 2.2
- **ADR-04 es posible gracias a esta decisión:** poner Account en Public Read Only tiene sentido porque Account *es* el cliente. Con particulares como contactos de una cuenta compartida, la pregunta ni se plantearía

Preguntas que un revisor hará

1. ¿Por qué no el modelo estándar Account = empresa, Contact = persona? → `Policy__c.Account__c` necesita un padre único, y una cuenta «Particulares» compartida destruye la propiedad del registro
2. ¿Se puede desactivar? → **No.** Nunca
3. ¿Cuánto almacenamiento cuesta? → Dos registros por persona. Hay que dimensionarlo antes de I1, no después
4. ¿Se verificó la compatibilidad con paquetes de AppExchange? → No hay ninguno previsto en el diseño. Si alguna vez se evalúa uno, la compatibilidad con Person Accounts es criterio de selección
5. ¿Cómo se evita contar dos veces en los reportes? → Criterio explícito en M1; entra en el checklist de evidencia de Fase 3

ADR-04 · Account en Public Read Only, pese al aislamiento por unidad

Estado	Aceptada
Fase	2.2 — Modelo de seguridad
Decisión en el log	SEC-04
Criterios que cumple	Contestable en revisión · **ADR-01 se apoya en ella**
Decisiones relacionadas	**ADR-01** (rollup en Apex), **ADR-03** (Person Accounts), HU-05 · HU-09 (aislamiento), «fuga por agregación» (FLS)

Contexto

HU-09 exige aislamiento por unidad de negocio: «*un agente de auto no ve las pólizas de vida*». El instinto, leyendo eso, es poner **todo** en Private.

Pero el caso negativo de HU-05 dice literalmente:

> «*Un agente de auto no ve las **pólizas de vida del mismo cliente***»

Esa frase sólo tiene sentido si el agente ve al cliente. Lo que está aislado es *qué tiene asegurado*, no *que exista*.

Y hay un requisito operativo que lo confirma: un Agente de Servicio tiene que **identificar a cualquiera que llame**, antes de saber de qué unidad es su póliza.

Alternativas evaluadas

1 · Account en Private — descartada

Rompe el contact center: el agente no puede identificar a quien llama ni encontrar al cliente para registrar el caso. Y plantea una pregunta sin respuesta: **¿por qué criterio se compartiría?** El cliente no pertenece a una unidad de negocio — puede tener pólizas en las tres. Una sharing rule sobre Account por `Business_Unit__c` no se puede escribir, porque Account no tiene unidad.

> Esa es la señal de que Private es el modelo equivocado: **cuando no existe criterio de compartición, es que el objeto no es lo que hay que aislar.**

2 · Account en Public Read/Write — descartada

Sin requisito que lo pida, y con un coste claro: cualquiera de los 65 podría modificar los datos de contacto de cualquier cliente.

3 · Account en Public Read Only — aceptada

Identidad visible para todos; **lo asegurado, aislado**.

Decisión

Account en Public Read Only. Contact queda en **Controlled by Parent** — que no es una decisión, es la consecuencia.

Lo que esto expone, dicho sin suavizar

Los 65 usuarios ven **la ficha de identificación y contacto de toda la cartera**.

Se acepta porque es el mismo requisito operativo que justifica la decisión, y porque **lo que no queda expuesto es lo que importa**:

	Protegido por
Qué tiene asegurado el cliente	Sharing rules sobre `Policy__c` por `Business_Unit__c`
Sus siniestros	Idem sobre `Claim__c`
Sus cotizaciones y oportunidades	OWD Private + propiedad del registro
Sus datos de salud	FLS régimen 2 + `PS_Unidad_Vida`

🔴 La fuga por agregación

Esta decisión crea un problema que sólo existe por ella, y que es **el mejor caso de FLS de todo el proyecto**.

`Total_Polizas_Activas__c` y `Suma_Primas_Vigentes__c` viven en Account, que es legible por todos. Un agente de auto que vea «4 pólizas activas» cuando sólo alcanza una **deduce que el cliente tiene pólizas en líneas que no puede ver**.

> **La protección a nivel de registro no se hereda en las sumas.** Un total agrega registros que el lector no puede abrir, y el total mismo es el dato que se filtra.

La compartición no sabe expresar eso: concede el registro entero o no lo concede. **Por eso ambos campos están ocultos por FLS** para Agente Comercial y Agente de Servicio, y visibles sólo para Gerente, Dirección y Administrador — el actor de HU-13 es el Gerente, y ninguna historia pide que un agente vea ese total.

Es también la corrección de una idea equivocada que duró varias fases: *«el mejor caso de FLS es la comisión»*. No lo es. **La comisión la protegen el acceso a objeto y la propiedad del registro**, no un permiso de campo. El mejor caso de FLS es éste, y existe por ADR-04.

Consecuencia que bloquea ADR-01

Con Account en Public Read Only, **`Policy__c` no puede ser hijo master-detail de Account** sin heredar ese OWD y destruir HU-09. Eso fuerza el lookup, el lookup descarta el Roll-Up Summary, y el descarte del Roll-Up Summary obliga a Apex.

> **ADR-04 es la raíz de ADR-01.** Un revisor que cuestione el OWD de Account está cuestionando, sin saberlo, por qué el rollup está en Apex.

Preguntas que un revisor hará

1. ¿No contradice HU-09? → No. HU-09 aísla *lo asegurado*; HU-05 exige que el cliente sea visible
2. ¿Por qué no Private? → No existe criterio de compartición: Account no tiene unidad de negocio
3. ¿Qué queda expuesto? → Identificación y contacto de toda la cartera. Nada de lo asegurado
4. ¿Y los totales de cartera no delatan lo que está aislado? → Sí: es la fuga por agregación, resuelta con FLS, y probada en **T13**
5. ¿Esto afecta a otras decisiones? → Sí: es la razón de que el rollup de HU-13 esté en Apex (**ADR-01**)

ADR-05 · El re-login no actualiza los accesos por unidad

Estado	Aceptada
Fase	2.5 — Identidad y SSO
Decisión en el log	IDE-09
Criterios que cumple	Contestable en revisión · el enunciado pidió justificarla por escrito
Decisiones relacionadas	IDE-03 (frontera de autoridad), IDE-04 (el IdP es autoritativo), IDE-08 (reporte de reconciliación), IDE-15 (reasignación de cartera), AOQ-11 (agente multi-unidad)

Contexto

HU-29 cubre el empleado que **cambia de unidad de negocio**. Su redacción parece exigir automatización — *«para no gestionar permisos de forma manual»* — pero su primer criterio dice otra cosa:

> *«Se define y documenta **si** el re-login actualiza los accesos automáticamente **o** requiere intervención.»*

Ese criterio se cumple documentando cualquiera de las dos opciones. Exige una decisión, no un resultado. Está muy bien escrito, y es lo que permite conservar AOQ-11.

Hecho de plataforma: JIT vía SAML mapea atributos a **Perfil y Rol**. **No asigna permission sets sin un handler Apex.**

Alternativas evaluadas

	Handler Apex de JIT	**Asignación manual ✅**
Multi-unidad (AOQ-11)	Necesita atributo multivaluado en el IdP — capacidad no verificada	Natural
Cobertura temporal	El IdP no sabe de vacaciones	Permission set **con fecha de expiración**

🔴 **Si falla**	**Código a medida en el camino de login: nadie entra**	Un alta tarda una hora más
Auditoría	Hay que programarla	Setup Audit Trail nativo + reporte de IDE-08

La tercera fila es la que decide

> Un handler de JIT es **código personalizado en la ruta de autenticación**. Un bug ahí no degrada una funcionalidad: **tumba la plataforma**. Para 65 usuarios en un Release 1, el riesgo no se paga.

Y hay un segundo argumento, estructural

El atributo de unidad de negocio del directorio es **de un solo valor**. Un agente que cubre auto **y** vida no cabe en un `department`.

Se elige **texto concatenado** (`auto;vida`) **interpretado por el administrador** — no parseado por código, precisamente porque la asignación es manual.

> **Cuando dos argumentos independientes apuntan a la misma decisión, deja de ser una preferencia.**
>
> El primero es de riesgo; el segundo es estructural: el directorio **no puede expresar** la multi-unidad de forma que un handler la consuma sin ambigüedad.

Decisión

Los accesos por unidad son manuales. El re-login no los actualiza.

El IdP gobierna usuario, perfil y rol; Salesforce gobierna los permission sets de unidad (IDE-03).

La respuesta a HU-29 es mixta, y hay que decirlo así

Al cambiar de unidad en el IdP	
Perfil y rol	Se actualizan **solos** en el siguiente login — IDE-04

Accesos por unidad (permission sets, grupos públicos)	**No** se actualizan — IDE-09
Cartera de registros	**No** se mueve sola — IDE-15

Leídos por separado, IDE-04 e IDE-09 parecen contradecirse. No lo hacen: es la frontera de IDE-03 aplicada al cambio y no al alta. **Lo que el IdP sabe, el IdP lo actualiza; lo que sólo Salesforce sabe, lo actualiza un administrador.**

Consecuencias

El control es el reporte de reconciliación (IDE-08), que compara lo que dice el IdP contra lo que Salesforce concedió. Sin él, «asignación manual» significa «esperamos que alguien se acuerde», y el tercer criterio de HU-29 —*«queda registro auditable del cambio de acceso»*— no se cumple.

⚠ Ese reporte necesita **línea base esperada por rol, no comparación 1:1**. El directorio dirá *«comercial»*, no *«auto + hogar + vida»*: los mandos tienen las tres unidades por diseño (IDE-10), y el reporte sólo debe marcar divergencia en los agentes.

Coste aceptado: cada alta exige **dos asignaciones manuales** —permission set (campos) y membresía de grupo público (registros)— y **nada las mantiene atadas**. El desajuste silencioso es RSK-SEC-02, y el peligroso es el segundo: grupo sin permission set, porque el usuario ve los siniestros con los campos en blanco y **puede no darse cuenta de que le falta información**.

Asimetría a vigilar: Salesforce sabe caducar un permission set; **no sabe caducar la membresía de un grupo público**. Es el escenario que acumula permisos (RSK-SEC-01).

HU-11 criterio 4, satisfecho a medias: *«no crea un usuario a medias»* es nativo — JIT estándar falla sin crear nada. *«Mensaje interpretable»* es parcial: el error SAML nativo es genérico. Se cubre con **guía de diagnóstico documentada** en lugar de con el handler que acabamos de descartar. Meter código en el camino de login para mejorar un texto sería invertir el argumento que sostiene esta decisión.

Evolución prevista: el handler Apex vuelve a la mesa si el volumen de cambios de unidad lo justifica. No es una puerta cerrada, es una puerta que hoy no compensa abrir.

Preguntas que un revisor hará

1. ¿HU-29 no pedía automatizar? → Su primer criterio pide **documentar la decisión**, no automatizar. Está citado literalmente arriba
2. ¿Y si el volumen de cambios crece? → El handler Apex es la evolución prevista; el umbral es de negocio, no técnico
3. ¿Cómo se audita una asignación manual? → Setup Audit Trail nativo + reporte de reconciliación de IDE-08 con línea base por rol
4. ¿Por qué no SCIM? → Automatizaría el desaprovisionamiento, pero depende de capacidades del IdP sin verificar. Queda como consideración posterior (IDE-05)
5. ¿Qué pasa si alguien asigna el grupo y olvida el permission set? → RSK-SEC-02. Mitigado con lista de verificación de alta en el runbook; **no eliminado**

Registro completo de decisiones

Las ~70 decisiones del proyecto viven en los registros por subfase del documento de Fase 2:

Prefijo	Subfase	Dónde
Modelo de datos	2.1	Diccionario y tabla de relaciones
SEC	2.2	Capas 1-9 + riesgos abiertos
UX	2.3	Decisiones registradas en 2.3
AUT · SLA	2.4	Decisiones registradas en 2.4
IDE	2.5	Decisiones registradas en 2.5
INT	2.6	Decisiones registradas

Este anexo no los sustituye: **desarrolla cinco de ellas.**